Detecting Shrinkflation in Canadian Grocery Prices

CMPT 353 Project Report

1. The Problem

This project asks whether Canadian grocery products are shrinking in package size while price holds steady or rises ("shrinkflation"), and whether that effect differs by grocery chain. The starting idea, from the course's own project suggestions, was broad: explore Project Hammer's scraped Canadian grocery price data. Refining it into a testable question required first understanding a structural constraint in the data that isn't obvious from the schema alone: each product's `id` in Project Hammer is a fixed identity whose package size never changes over its lifetime. A retailer that shrinks a product's package size does so by discontinuing the old listing and creating a new one, not by editing the existing one in place. That single fact reframed the whole problem: shrinkflation can't be detected by watching one id's size change over time, because that never happens in this data. It has to be detected by recognizing when one id disappears and a smaller-size id for what is recognizably the same product takes its place, within the same vendor, in a short window of time. That reframing is the actual technical core of this project, everything downstream (the name-matching step, the timing-gap filter, the plausibility bounds) exists to make that recognition reliable rather than noisy.

A secondary question, added once the primary result was established, was whether our own price-per-unit trend, built independently from this data, tracks the official StatCan Food Consumer Price Index over the same period, and if not, why not.

2. The Data

2.1 Sources

- Project Hammer (jacobfilipp.com/hammer): scraped daily prices from 8 Canadian grocery chains (Loblaws, No Frills, Metro, Voila, T&T, Walmart Canada, Save-On-Foods, Galleria), covering February 2024 to August 2026. Two tables: product.csv (one row per product, ~305,000 rows) and raw.csv (one row per price observation, ~70 million rows once parsed, 3.5GB; the file has ~75 million physical lines, but some records contain embedded newlines and span more than one).
- Statistics Canada Table 18-10-0004-01, Consumer Price Index, monthly, not seasonally adjusted, filtered to the Food category and Canada geography, covering the same window.

2.2 Cleaning

The raw data required substantial cleaning before any analysis was possible, and several of the issues found were not visible until specific downstream steps exposed them. This is a fair summary of what real scraped data looks like, not a minor housekeeping step.

- Corrupted vendor field (4 rows): a URL or a stray timestamp had leaked into the vendor column from a shifted-field scraping error. Dropped.
- "error" placeholder product names (3,191 rows): scraper failures recorded as literal products, not real listings. Dropped.
- Invalid dates (11 rows): a handful of `nowtime` values appeared as the Julian day zero sentinel (-4713-11-24) instead of a real timestamp. Dropped.
- Prices stored 100x too large (42,578 rows, roughly 0.06% of price rows): current_price was recorded in cents rather than dollars for a subset of rows, in two different source notations ("$X.XX" and "X¢"). Detected by cross-referencing current_price against the retailer's own price_per_unit field for the same row, and corrected.
- Implausible price-per-unit values (a further small tail): a handful of rows had a price_per_unit far below one tenth of a cent per gram, corrupted rather than real. These were caught downstream, in the shrinkflation-matching and price-index code specifically, since the corruption only became visible once paired against a sibling product or a neighboring month.

One limitation is worth stating plainly here rather than in a closing caveats list: the cents-bug detection above requires an internal inconsistency to be visible within a single row (current_price disagreeing with price_per_unit). A rarer variant, where both fields are inflated by the same factor together, produces a row that looks internally consistent and is not caught by this check. It was only found by chance, when a corrupted row happened to sit next to an uncorrupted sibling product during the shrinkflation-matching step. The true count of this class of error is therefore unknown and likely larger than what was caught.

3. Techniques

Each stage below is implemented as tested, version-controlled code, not a one-off script; the project has 78 automated tests covering the parsing, matching, cleaning, and statistical logic.

3.1 Package size parsing

Product package sizes are embedded in free-text strings (e.g. "500g", "2 x 500g", "148gx4"), not a structured field, and about 18% of the messiness is unrecoverable (garbage listings with no size information at all, or a decoy number like a fat percentage with no unit attached). A vectorized regex parser (three compiled patterns: a multipack "count x quantity unit" form, its reverse "quantity unit x count", and a plain single quantity+unit form) extracts and normalizes quantities to grams, millilitres, or item counts. It recovers a usable size for 82.0% of products, validated against the real data, not just synthetic test cases.

3.2 Product matching across listings

The same physical product is described with different, inconsistent text across different id listings (reordered words, punctuation differences, added brand prefixes). Matching uses TF-IDF vectorization plus cosine similarity (scikit-learn), with the package size stripped out of the text before comparing. Similarity alone is not sufficient, "Kraft Peanut Butter Crunchy 500g" and "...Smooth 500g" score highly similar since they share nearly every word, so a veto list of qualifier words (crunchy/smooth, original/spicy, salted/unsalted, etc.) blocks a match when exactly one side has such a word and the other doesn't. The similarity threshold (0.9) was not chosen arbitrarily: it was validated against real labeled pairs pulled from the data, both confirmed true matches and confirmed different-product pairs, to find where the score distributions actually separated.

3.3 Shrinkflation detection

For each vendor separately, every product is matched against the rest of that vendor's own catalog (excluding itself) using the same TF-IDF approach. A candidate pair is kept only if: the matched pair has different ids, the same unit type (both grams or both millilitres), a real size decrease, a plausible size ratio (excluding corrupted extreme values, such as a source value of "1g" for a product that should weigh 500g), and a clean timing handoff, the outgoing id's last active date and the incoming id's first active date within 14 days of each other, distinguishing an actual replacement from two listings that simply coexisted.

3.4 Statistical testing

Every test below followed the same discipline: check the relevant assumption first (normality via `scipy.stats.normaltest`, equal variance via `levene`), then let that result determine which test to run, rather than assuming a test in advance.

- Primary comparison: is price-per-unit change larger for shrink-candidate pairs than for a control group of same-vendor, same-name, same-size matched pairs (no real size change)? Both groups failed the normality check decisively, so a Mann-Whitney U test was used, one-sided.
- Per-chain comparison: does the size of that price-per-unit change differ across the 8 vendors? Kruskal-Wallis, the non-parametric alternative to one-way ANOVA, again chosen because the normality check failed.
- Regression: what predicts the magnitude of price-per-unit change? A linear regression and a random forest regression (scikit-learn), using size_change_pct, the product's starting price-per-unit and quantity, vendor, and unit as features, with a train/test split to check for overfitting. The initial unconstrained forest overfit badly (train $R^2 = 0.89$, test $R^2 = 0.085$), which the train/test comparison caught. The extreme tail of the target mixes genuine large price changes with residual data artifacts that cannot be separated reliably

(corrupted prices produce changes in the hundreds of thousands of percent), so the models were fit on the central range of the target (within 1.5× the interquartile range) and the forest was depth-limited; the reported fit applies to that central range, and predictive power remains modest either way.

3.5 Independent price index

A second, independent measure of the same question: build a price index from Hammer data alone and compare its trend to StatCan's official Food CPI over the same window. Rather than a fixed base month (which would lose most products within a few months, since the product basket in this data turns over constantly), the index chains together month-over-month price relatives, using whichever products have data in each adjacent pair of months, the same idea StatCan itself uses to handle a changing consumer basket. Each monthly link is the geometric mean of the price relatives (a Jevons index, the formula StatCan uses at this level), a choice that section 4.4 shows matters enormously: the arithmetic mean drifts upward whenever prices bounce, and scraped grocery prices bounce constantly with sales. The index was rebased to start in June 2024 rather than February 2024, once it was discovered that Hammer's scraping operation was still ramping up in its first four months (February 2024 had 2,296 price observations; June 2024 had 1.37 million), making the earliest months an unreliable anchor for a chained index.

4. Results

4.1 Detected shrinkflation candidates show substantially larger unit-price increases

2,214 shrinkflation candidates were identified across the 8 chains. Median price-per-unit change for these pairs was +19.0%, compared to +0.3% for the 49,209 control pairs with no size change (Mann-Whitney U, one-sided, $p < 0.001$). 87.7% of shrink-candidate pairs saw price-per-unit rise, versus 44.7% of control pairs.

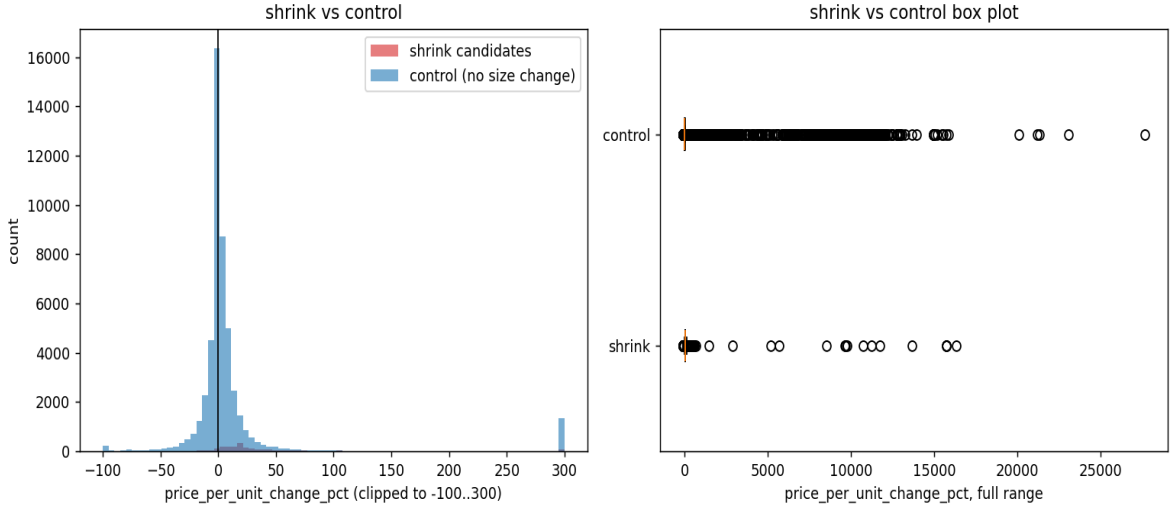


Figure 1. price_per_unit_change_pct for shrink-candidate pairs (red) versus same-size control pairs (blue). The shrink distribution is visibly shifted right of the control distribution.

4.2 The effect varies significantly by chain

A Kruskal-Wallis test across the 8 vendors' shrink-candidate price-per-unit changes was significant ($p \approx 4.1 \times 10^{-7}$). The size of the effect ranged from Save-On-Foods at a median +31.5% down to No Frills and Galleria at roughly +13%, though Galleria's sample is small (n=27) and should be read cautiously.

Chain	Median price-per-unit change	n
Save-On-Foods	+31.5%	139
T&T	+26.8%	175
Metro	+25.7%	379
Walmart	+25.6%	344
Voila	+18.6%	210

Loblaws	+17.7%	711
No Frills	+13.5%	229
Galleria	+12.3%	27

4.3 What predicts the size of the price change

The regularized random forest reached a modest R^2 (train 0.068, test 0.066, no longer overfit once regularized), meaning most of the row-to-row variation in exactly how much price-per-unit changes is not explained by these five features, itself a real and worth-stating finding about how idiosyncratic individual retail pricing decisions are. Among the features that do matter, size_change_pct dominates (58% of the forest's feature importance), consistent with the significance test above, followed by the product's starting price-per-unit (12%) and vendor identity (spread across the vendor dummy variables).

4.4 The index formula alone was worth 12 percentage points

The first version of the index averaged each month's price relatives arithmetically (a Carli-style index) and rose to 113.7 by June 2026 (+13.7%), more than double CPI's +6.5% over the same 24 months. That headline was an artifact of the formula, not of grocery prices: the arithmetic mean of price relatives drifts upward whenever prices bounce (a price that halves on sale and rebounds averages to +25% despite ending where it started), and scraped grocery data is full of exactly this pattern. Switching each monthly link to the geometric mean, the Jevons formula StatCan itself uses and under which a bounce nets out to zero, collapsed total growth to +1.8%. The official +6.5% sits between the two constructions.

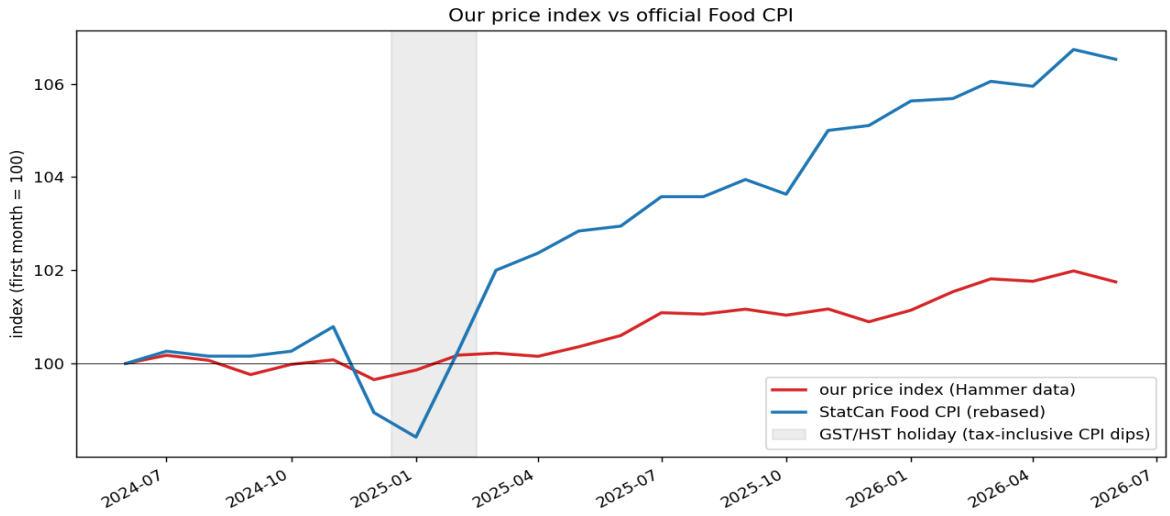


Figure 2. Our price-per-unit index (red, geometric/Jevons version) versus StatCan's official Food CPI, both rebased to 100 at June 2024. The shaded band marks the federal GST/HST holiday (December 14, 2024 to February 15, 2025), which shows up only in the tax-inclusive CPI.

The remaining gap plausibly reflects construction differences rather than real disagreement: our index weights every scraped product equally at its listed price, sales included, over a basket with heavy turnover, where CPI weights by actual household spending. A sensitivity check restricted to the 79 products scraped in all 31 months showed near-zero growth, consistent with the lower figure. The dip in CPI inside the shaded band is the federal GST/HST holiday: CPI measures tax-inclusive prices, so the temporary tax removal registers as a price drop, while our tax-exclusive shelf-price index correctly stays flat through the same window.

5. Limitations

- The dual-inflation variant of the cents bug (both current_price and price_per_unit scaled together) is not fully caught; the true count of affected rows is unknown.

- The shrinkflation detection method is conservative by construction (a clean name match, a 14-day timing window, a plausible size ratio all have to hold), so the 2,214 candidates found are very likely an undercount of the true number of shrinkflation events, not an overcount.
- The result shows correlation, a real, significant, and large association between a size decrease and a price-per-unit increase, not proof of retailer intent or strategy. Other explanations (ingredient cost inflation coinciding with a resize) are not ruled out by this analysis alone.
- No product category dimension exists in this data (only vendor), so the question of whether the effect differs by food category rather than by chain could not be tested.
- The window covered is short (30 months, all that Project Hammer has scraped), and online-only, so it cannot speak to longer-term trends or to any gap between online and in-store pricing.
- The price index is highly sensitive to construction choices: the formula alone moves the answer by 12 percentage points, and equal weighting, sale prices, and basket turnover all push it further from what CPI measures. Its specific growth figure should be read as one point in a bracket around the truth, not a precise measurement.

Project Experience Summary

- Designed and built an end-to-end data pipeline in Python (pandas, scikit-learn, scipy) to detect shrinkflation across 70 million scraped Canadian grocery price records from 8 retail chains, applying TF-IDF text matching and non-parametric hypothesis testing, finding that products relaunched at smaller sizes saw a median 19.0% price-per-unit increase versus 0.3% for otherwise-identical stable products ($p < 0.001$).
- Diagnosed and corrected systematic data-quality defects in a raw 3.5GB scraped dataset, including a 100x pricing error affecting over 42,000 records, by designing cross-field validation logic that catches internal inconsistencies between a record's price and its stated price-per-unit.
- Built the pipeline test-first (78 automated tests) and caught my own methodological errors through independent verification, including an overfit regression model exposed by train/test comparison and a 12-percentage-point upward bias in a self-built price index from arithmetic rather than geometric averaging of price relatives, validated against Statistics Canada's official Consumer Price Index.